

08 The Application Layer

08.03 World Wide Web

The World Wide Web

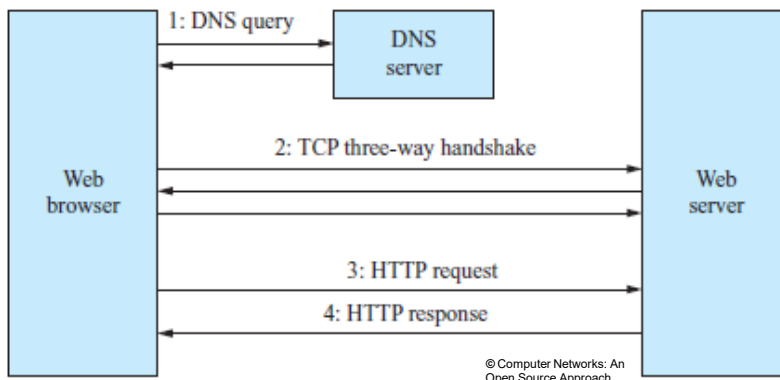
- Architectural Overview
- Static Web Documents
- Dynamic Web Documents
- HTTP – The HyperText Transfer Protocol
- Performance Enhancements
- The Wireless Web

The World Wide Web

- Started by Tim Berners-Lee as a project at the European Organization for Nuclear Research (CERN), WWW has been the most popular medium for information retrieval of all sorts since 1989.
- By using a Web browser users can access any online Web pages by requesting it to a Web server

Architectural Overview

- How a Web client interacts with a Web server



Architectural Overview

- The Web is an information space formed with a great quantity of Web pages and documents known as *resources*.
- Web naming is a mechanism for naming resources on the Web, while Web addressing provides ways to access the resources.
- Uniform Resource Identifiers (**URIs**) are short strings that identify resources in the Web through a variety of naming schemes and access methods.
- Uniform Resource Locators (**URLs**) are a subset of URIs to describe the addresses of resources accessible on the Web.
- A Uniform Resource Name (**URN**) is a name of global scope which does not imply a *location*.
- Note that URLs are used for *locating* or finding resources, whereas URNs are used for *identification*.

URLs – Uniform Resource Locaters

Some common URLs.

Name	Used for	Example
http	Hypertext (HTML)	http://www.cs.vu.nl/~ast/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Local file	file:///usr/suzanne/prog.c
news	Newsgroup	news:comp.os.minix
news	News article	news:AA0134223112@cs.utah.edu
gopher	Gopher	gopher://gopher.tc.umn.edu/11/Libraries
mailto	Sending e-mail	mailto:JohnUser@acm.org
telnet	Remote login	telnet://www.w3.org:80

HTTP

- HTTP is an application layer protocol used for the transfer of Web resources, whether these are documents, Web pages, or images.
- Like most application-layer protocols on the Internet, HTTP uses a request response–based communication scheme.
- Although the request-response mechanism occurs directly between server and client, it is possible that additional systems in the middle participate in the message exchange. Such systems include proxies, Web caches, gateways, or other systems

HTTP Methods

The built-in HTTP request methods.

Method	Description
GET	Request to read a Web page
HEAD	Request to read a Web page's header
PUT	Request to store a Web page
POST	Append to a named resource (e.g., a Web page)
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Reserved for future use
OPTIONS	Query certain options

HTTP Responses

The status code response groups.

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

HTTP Message Headers

Some HTTP message headers.

Header	Type	Contents
User-Agent	Request	Information about the browser and its platform
Accept	Request	The type of pages the client can handle
Accept-Charset	Request	The character sets that are acceptable to the client
Accept-Encoding	Request	The page encodings the client can handle
Accept-Language	Request	The natural languages the client can handle
Host	Request	The server's DNS name
Authorization	Request	A list of the client's credentials
Cookie	Request	Sends a previously set cookie back to the server
Date	Both	Date and time the message was sent
Upgrade	Both	The protocol the sender wants to switch to
Server	Response	Information about the server
Content-Encoding	Response	How the content is encoded (e.g., gzip)
Content-Language	Response	The natural language used in the page
Content-Length	Response	The page's length in bytes
Content-Type	Response	The page's MIME type
Last-Modified	Response	Time and date the page was last changed
Location	Response	A command to the client to send its request elsewhere
Accept-Ranges	Response	The server will accept byte range requests
Set-Cookie	Response	The server wants the client to save a cookie

Exaple HTTP Usage

```
Trying 4.17.168.6...
Connected to www.ietf.org.
Escape character is '^]'.
HTTP/1.1 200 OK
Date: Wed, 08 May 2002 22:54:22 GMT
Server: Apache/1.3.20 (Unix) mod_ssl/2.8.4 OpenSSL/0.9.5a
Last-Modified: Mon, 11 Sep 2000 13:56:29 GMT
ETag: "2a79d-c8b-39bce48d"
Accept-Ranges: bytes
Content-Length: 3211
Content-Type: text/html
X-Pad: avoid browser bug
```

```
<html>
<head>
<title>IETF RFC Page</title>
```

The start of the output of
www.ietf.org/rfc.html.

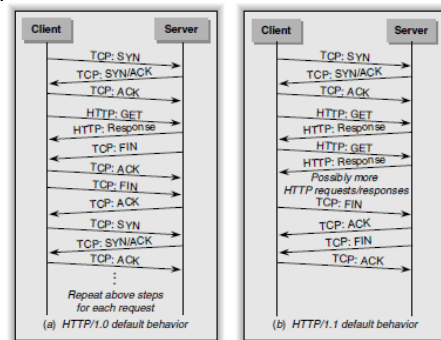
```
<script language="javascript">
function url() {
  var x = document.form1.number.value
  if (x.length == 1) {x = "000" + x }
  if (x.length == 2) {x = "00" + x }
  if (x.length == 3) {x = "0" + x }
  document.form1.action = "/rfc/rfc" + x + ".txt"
  document.form1.submit
}
</script>
</head>
```

HTTP 1.1 Improvements

- Most important changes were made to improve the average response time, which is achieved by allowing multiple transactions to take place over a single, persistent TCP connection.
- Delivery of dynamically generated pages has been improved by supporting chunked transfer encoding, which allows a response to be sent before its total length is known.
- Other additions allow multiple domains to be served from a single IP address, enabling a more efficient use of the existing IP address space.

HTTP: persistent connections

- In HTTP 1.0 and earlier versions, the client has to establish a new, separate TCP connection for each resource to be retrieved.
- HTTP 1.0 client has to open and close separate TCP connections for each of the objects that resides on the web server, which results in increased access delays and in additional protocol overhead.
- Much can be saved in HTTP/1.1 with a default behavior allowing multiple requests and responses to be sent through a single *persistent connection*.

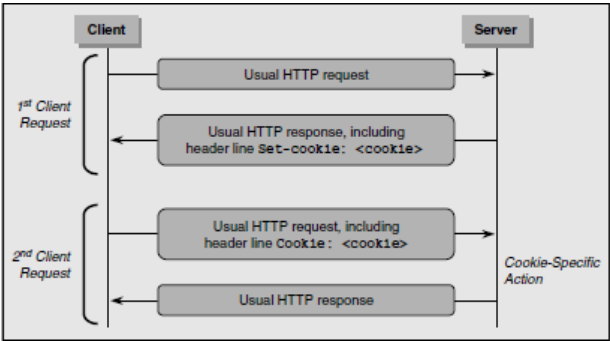


© HOFFMAN and BEAUMONT -
Content Networking
•Architecture, Protocols, and
Practice

Statelessness and Cookies

- HTTP is a stateless protocol, which means that the server does not retain any information across multiple client requests—the server generates a response and forgets about the client and its request (except for considering TCP-level information that needs to be maintained for persistent connections).
- Some sites, although, needs to keep track of the clients operations.
- The provider might also be interested to learn whether a customer has visited the site before, so that appropriate content can be presented.
- Cookies are HTTP's way to manage such information, shifting the burden of state maintenance from the server to the client.
- A cookie is a small amount of state information that a Web server sends to a Web client for storage and presentation with future requests.

Statelessness and Cookies



Domain	Path	Content	Expires	Secure
toms-casino.com	/	CustomerID=497793521	15-10-02 17:00	Yes
joes-store.com	/	Cart=1-00501;1-07031;2-13721	11-10-02 14:22	No
aportal.com	/	Prefs=Stk:SUNW+ORCL;Spt:Jets	31-12-10 23:59	No
sneaky.com	/	UserID=3627239101	31-12-12 23:59	No

Some examples of cookies.

Pros of stateless servers

- Chief benefit: Potential for replication
 - Improved performance: A sysadmin can fire up N copies of a website (on N machines) and any machine can serve each request.
 - Improved reliability: If a machine crashes, then another can be started up in its place, and no data gets lost in the process.

Cons of stateless servers

- Chief problem: Keeping track of which requests "go together"
 - Security challenge: If a user submits username & password in http request X, then tries to access resources in http request Y, how does the server know that request Y is from somebody who already logged in?
 - By the time that request Y comes in, the server will already have forgotten that request X ever occurred.
 - And on a replicated system, request Y might be served by a different machine than request X.

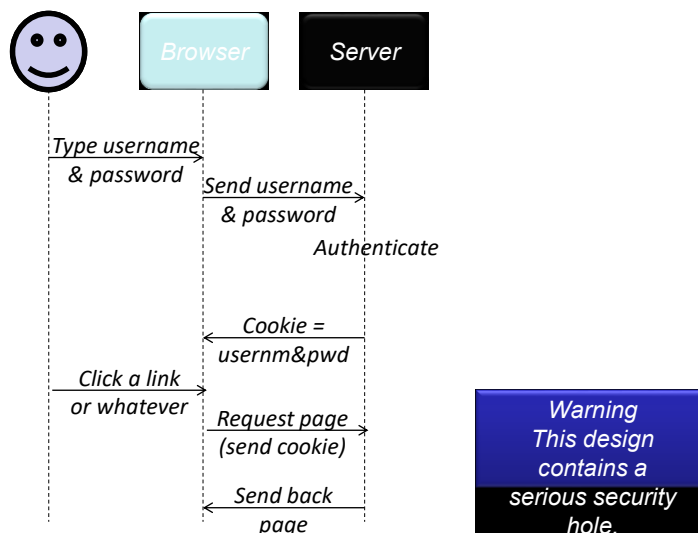
Cookies to the rescue!

- Reminder:
 - Cookie = a piece of data that is automatically copied between the client and server
- Cookies can be set by the client (as in the last unit) *or* by the server.

A simple way to use cookies for login...

- When user sends a valid username & password in request X, the server replies with a cookie containing the username & password
- When user subsequently makes request Y, the browser sends along the cookie.
 - Sounds appealing: user only needs to log in once
 - Serious security hole: anybody who gets his hands on the user's computer can see cookies

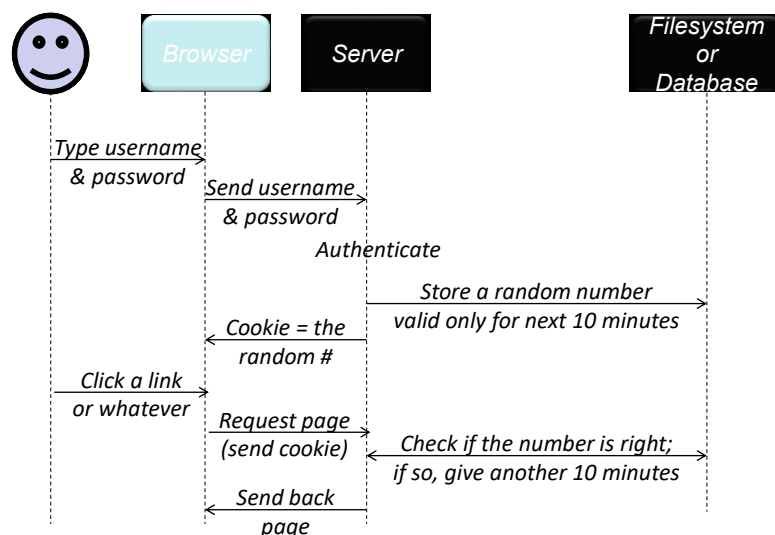
Using just cookies for login



A more secure way of cookies+login

- When user sends a valid username & password in request X, the server replies with a cookie containing a secret that the client couldn't possibly have guessed.
- When user subsequently makes request Y, the browser sends along the cookie.
 - Since the client couldn't have guessed this value without logging in, the server knows that the user did in fact previously log in.

Using cookies for login



Session = state stored across requests

- This is what we call a "session"
- Session is basically an *add-on* to the basic http functionality of a website
 - So that the website *can* remember information across requests.
- You can store lots of stuff in session
 - Numbers, strings, stringified objects, ...

Pros of sessions

- Stores information between requests
- Much more secure than the simple cookie-based approach I showed you
 - A bad person would need to steal the random number (cookie) within 10 minutes of its creation

Cons of sessions

- Requires your web server to have write-access to some sort of storage medium
 - File system, database, ..., if you want replication
 - Otherwise just use memory (lost on server crash)
- Requires user to access site every few minutes
 - Though you can configure longer or shorter times
 - This is a tradeoff between usability & security.

Summarizing cookies vs sessions

- Cookies
 - Little bits of data that are stored on client but also copied automatically to the server
 - Useful for storing little bits of data on the client, but they are visible to everybody
 - So don't store sensitive data in cookies
- Sessions
 - Data is stored on the server (e.g., filesystem), keyed by a random number
 - The random number is sent as a cookie to the browser
 - And the random number expires after a little while

When to use cookies vs sessions

- Use cookies when
 - You need to save a small amount of data between requests, and it doesn't need to be kept secret
- Use sessions when
 - You need to save a larger amount of data between requests, or when the data needs to be secret

Examples of information not to store in unencrypted cookies

- Passwords
- Credit card numbers
- Social security numbers
- Student ID numbers
- Birthdates
- List of diseases the user has contracted
- Anything that must be kept secret

Cookies and GDPR

- **GDPR applies** if you're:
 - **Offering goods and services** to people in the EU
 - **Profiling people** in the EU
- These are exactly the what **cookies do**
- Article 12 of the GDPR:
 - you're required to provide information about all the personal data you process "***in a concise, transparent, intelligible and easily accessible form, using clear and plain language.***"
- Do a cookies policy

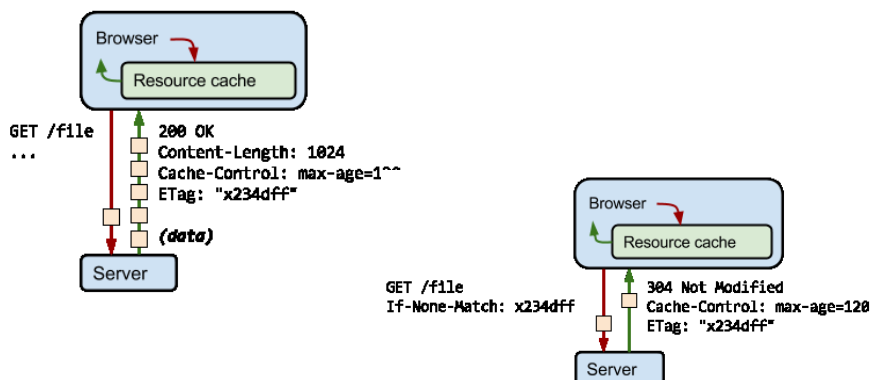
Cookies policy GDPR compliants

- Explains what cookies **are** and what they **do**;
- Explains **how you use** cookies, for example to improve your website or advertise to your users;
- Gives details of any **third-party cookies** that your users might encounter on your site;
- Explains your use of **analytics and remarketing**, and the privacy implications of these tools;
- Gives details of any other **tracking technologies** you use such as web beacons;
- Provides a **full list** of the cookies you use on your website;
- Tells users how they can **control their consent** over the various first and third-party cookies on your website.

Web Caching

- Web caching is a mechanism to expedite document downloading on the World Wide Web.
- Just like the ordinary caching concept in computer systems, a copy of remote content previously retrieved by users is kept locally in the cache server for future access to improve bandwidth efficiency and, most importantly, the responsiveness of the surfing experience.
- Upon receiving a request, the cache server checks whether a valid copy is present.
- If yes (cache hit), then the server returns the cached page immediately to the client; otherwise the client (browser) will receive the message Page Unfound.
- The client browser then continues the quest of retrieval by sending a request directly to the Web server, bypassing the cache server

Browser HTTP Caching



Transparent Proxy

- A cache server can also act as a proxy server, which helps forward queries to proper destinations if a cache miss occurs.
- The forwarding destination could be other cache servers or the corresponding Web server.
- This feature is beneficial in two respects:
 1. First, the overhead of resending the request from the client to the Web server is eliminated.
 2. Second and most importantly, the utmost control of the network can be achieved by concentrating all the accesses to the proxy server and monitoring them.
- Normally the Web caching feature requires that the client preconfigure the browser to make sure it checks with the cache server in the first place specifying its address.

HTTP/2 : why

- HTTP/2 is the first new version of HTTP since HTTP 1.1
- The HTTP/2 specification was published as RFC 7540 in May 2015
- Performance matters
 - HTTP/2 reduces the impact of latency on web applications
- TLS is becoming the default
 - HTTP/2 amortizes TLS costs for the entire application
- Enabling new web development

HTTP/2 : goals

- User perceivable improvement in web site performance
 - Decrease latency to improve page load speed in web browsers :
 - Data compression of HTTP headers
 - HTTP/2 Server Push
 - Pipelining of requests
 - Fixing the head-of-line blocking problem in HTTP 1.x
 - Multiplexing multiple requests over a single TCP connection
- Work with today's internet
- Remain compatible with existing content

HTTP/2: differences from HTTP 1.1

- HTTP/2 leaves most of HTTP 1.1's high-level syntax, such as methods, status codes, header fields, and URIs, the same. What's new is how the data is framed and transported between the client and the server.
- Performance improvements in the first draft of HTTP/2 come from multiplexing of requests and responses to avoid the head-of-line blocking problem in HTTP 1 (even when HTTP pipelining is used), header compression, and prioritization of requests

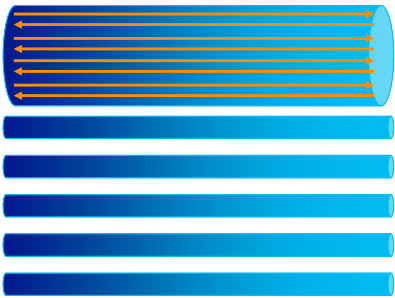
HTTP/2: differences from HTTP

1.1

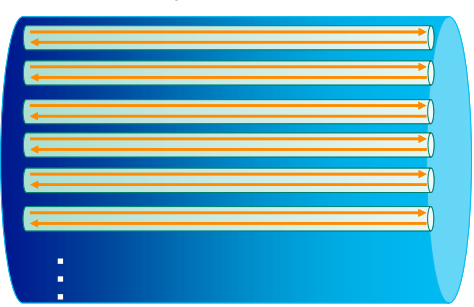
- Websites that are efficient minimize the number of requests required to render an entire page by minifying (reducing the amount of code and packing smaller pieces of code into bundles, without reducing its ability to function) resources such as images and scripts.
 - Minification is not necessarily convenient nor efficient and may still require separate HTTP connections to get the page and the minified resources.
- HTTP/2 allows the server to "push" content, that is, to respond with data for more queries than the client requested. This allows the server to supply data it knows a web browser will need to render a web page, without waiting for the browser to examine the first response, and without the overhead of an additional request cycle.

HTTP/2 Connections and Streams

HTTP/1.1 – Request = Connection



HTTP/2 – Request = Stream



Each request required dedicated TCP connection	TCP connection can have multiple streams (requests)
Responses come in order per connection	Responses can come out of order, server can optimize
Each connection requires setup + slow start	No connection setup for new streams, no slow start
Application sees "connections"	Streams are represented as "connections" to apps

HTTP/2: Header Compression and Binary Encoding

- In HTTP/1.x, header data is sent as plain text. In HTTP/2, header data for the single, multiplexed connection is compressed according to the new HPACK standard, which uses Huffman coding.
- Header data is also sent in binary format, further reducing file size.
- Header compression is not always a big positive. Compressing header data adds CPU overhead.
- In cases where the uncompressed request header fits in a single TCP packet, compression doesn't help.
- For responses, header data is often much smaller than the accompanying data, so the benefit of header compression is small as well.

HTTP/2: Header Compression and Binary Encoding

- Both compression and binary encoding make for obscurity in debugging, compared to plain text.
- In HPACK compression, a static table contains known header fields and common values, each assigned an index number.
- As header transmission starts, values in the static table replace text strings in the header.
- As header transmission continues, a dynamic table is built up as well, assigning codes to new header-value pairs.
- The dynamic table is used as fields repeat within the session.

HTTP/2 Push

- HTTP/2 push allows the server to send data to the browser without having to wait for the browser to explicitly request it first.
- This is useful because normally we need to wait for the browser to parse the .html file and discover the needed links before these resources can be sent.
- Push also helps to ensure better bandwidth utilization at the start of the connection.

HTTP/2 Push

